NATURAL LANGUAGE PROCESSING

PROJECT 3

RESUME PARSER & CANDIDATE MATCHER

Named Entity Recognition • Regex Information Extraction
Skill Matching • Candidate Ranking

STUDENT AND FACULTY LEARNING MANUAL

Beginner-friendly concepts • Complete Python source code • VS Code execution
Troubleshooting • Sample resumes • Experiments • Viva questions • Faculty teaching plan

Prepared from the NLP Projects specification and the resume_parsing files used in the classroom execution session.

How to use this manual

This manual is intended for students and faculty who want to understand and execute the Resume Parser & Candidate Matcher from the beginning. Follow the sequence: understand the problem, verify the Python environment, run the Streamlit interface, examine the results, and then study the code. The core demonstration uses three built-in sample resumes. Uploaded files are supported as UTF-8 text files (.txt).

Part	What it covers
1. Project overview	Problem statement, learning outcomes and processing workflow
2. Concepts	NER, spaCy, regular expressions, skill vocabulary, sets and matching score
3. Environment	Folder structure, dependencies and exact PowerShell commands used
4. Application use	Streamlit input, resume upload, results table and extracted details
5. Complete code	Full app.py, parser.py, API, Elasticsearch helper and sample data
6. Code walkthrough	How each component works and how data moves through the program
7. Results and limitations	Sample score calculation, caveats and responsible classroom use
8. Troubleshooting	Missing spaCy, missing model, file decoding, blank input and server issues
9. Experiments	Suggested tests and observation worksheet
10. Assessment	Student exercises, viva questions and faculty teaching plan

1. Project overview

Project title: Resume Parser & Candidate Matcher (NER and Information Extraction).

Learning objective: Combine Named Entity Recognition, regular-expression pattern matching and set-based algorithms to convert resume text into structured information and compare candidate skills with a job description.

Learning outcome	What the learner should be able to do
Explain NER	Describe how spaCy identifies entities such as PERSON and ORG in text.
Extract contact details	Use regular expressions to identify email addresses and phone-number candidates.
Extract skills	Use a configurable vocabulary of canonical skills and aliases.
Extract education terms	Identify configured degree and qualification terms in resume text.
Calculate match scores	Use set intersection and a normalized skill-coverage ratio.
Rank candidates	Sort multiple resumes by their calculated skill-match scores.
Run a web interface	Use Streamlit to provide a job description, upload text resumes and inspect results.

Expected project outcome

A local Streamlit application that accepts a job description and one or more resume text files, extracts candidate details, calculates skill coverage and displays a ranked candidate table with matched and missing skills.

1.1 Project scope used in this manual

The classroom application runs locally and uses the spaCy model en_core_web_sm, a regular-expression extractor, and a rule-based skill vocabulary. The optional FastAPI and Elasticsearch files are included as extension code; they are not required to launch the Streamlit application.

Important scope boundary

The supplied Streamlit interface accepts .txt files only. It does not directly parse PDF or DOCX resumes. PDF/DOCX conversion, OCR, model training and production hiring workflows are outside the core demonstration.

2. Core concepts

2.1 What is resume parsing?

A resume is usually written as free-form text containing a person's name, contact information, education, work experience and skills. Resume parsing converts selected information from that unstructured text into fields that can be searched, displayed or compared.

2.2 Named Entity Recognition (NER)

Named Entity Recognition is an NLP task in which a model detects spans of text and assigns labels to them. In this project, spaCy processes the resume and the code collects entities with the labels PERSON and ORG.

Entity label	Meaning	Example in a resume
PERSON	A person or human name	Ananya Rao
ORG	An organization, company or institution	ABC Technologies
Other labels	Other entity types may also be detected by the model	The current parser does not store all entity labels.

The model does not understand a resume exactly as a human recruiter does. Entity detection is probabilistic and depends on the wording, formatting and language model. A name or organization can be missed or mislabeled, so extracted fields should be checked.

2.3 spaCy and en_core_web_sm

spaCy is the NLP library used to process resume text. en_core_web_sm is the small pretrained English pipeline installed for this project. The code loads it with spacy.load(SPACY_MODEL), where the default model name is en_core_web_sm. The model is loaded only once per Python process through a one-entry cache.

> **Model choice in the project documents**
>
> The main NLP Projects PDF mentions en_core_web_trf in its initial design description. The environment guide and the code used in the completed classroom run use en_core_web_sm. This manual documents the model that was actually installed and executed. The transformer-backed model is an optional later comparison, not a requirement for this run.

2.4 Regular expressions (Regex)

A regular expression is a pattern used to search text. This project uses regex to find email-like strings and phone-number candidates. Regex is suitable for structured patterns, while NER is used for contextual entities such as person and organization names.

Field	Extraction method	Important caveat
Name	spaCy PERSON entities	The first detected PERSON entity is displayed as the candidate name; verify it.
Email	Regex pattern	The pattern recognizes common email forms but is not a complete email validation service.
Phone	Permissive regex pattern	May capture unsuitable numeric strings; validate country-specific formats.
Organization / experience	spaCy ORG entities	Organizations are stored in the output field named experiences.
Skills	Configured aliases with phrase-boundary checks	Only skills listed in SKILL_ALIASES can be found.
Education	Configured degree-term vocabulary	This checks for listed terms, not full qualification semantics.

2.5 Skill vocabulary and aliases

The parser defines SKILL_ALIASES: a dictionary whose keys are canonical skill names and whose values contain alternative spellings or abbreviations. For example, machine learning can be detected from “machine learning” or “ml”; natural language processing can be detected from “natural language processing” or “nlp”.

```
SKILL_ALIASES = {
    "python": ["python"],
    "machine learning": ["machine learning", "ml"],
    "natural language processing": ["natural language processing", "nlp"],
    "scikit-learn": ["scikit-learn", "sklearn"],
    "pytorch": ["pytorch", "torch"],
    "fastapi": ["fastapi"],
    "docker": ["docker"],
}
```

The displayed example is an excerpt. The full editable vocabulary is included in the complete parser.py source later in this manual. Faculty can add domain-specific skills before using a different job description.

2.6 Python sets and skill matching

A set stores unique values. The intersection operator & returns elements that occur in both sets. For this project, one set contains skills detected in a candidate resume and the other contains skills detected in the job description.

```
matched_skills = set(parsed["skills"]) & set(required_skills)
match_score = len(matched_skills) / len(required_skills)
```

The result is a fraction between 0 and 1. The Streamlit interface displays it as a percentage by multiplying by 100. The denominator is the total number of skills detected in the job description by the configured vocabulary.

Worked example

If 6 of the 8 detected job skills are present in a resume, the score is 6 / 8 = 0.75, or 75%. This score represents configured-skill coverage only; it does not measure experience quality, skill proficiency, education equivalence or overall candidate suitability.

2.7 Data processing workflow

Stage	What happens
1. Job description	The user enters or edits job requirements in Streamlit.
2. Candidate input	The app uses uploaded .txt files or the three built-in sample resumes.
3. NER and regex	spaCy identifies PERSON and ORG entities; regex extracts emails and phone candidates.
4. Skill and education extraction	The vocabulary and education-term list are matched against the resume text.
5. Required-skill extraction	The same configured skill vocabulary is applied to the job description.
6. Match calculation	Set intersection determines matched skills; set difference determines missing skills.
7. Ranking and display	Candidates are sorted by descending match score and shown in a Streamlit table.

3. Project architecture and files

```
C:\Users\spriy\nlp_projects\
|
+-- nlp-env\                          # Shared Python virtual environment
|
+-- resume_parsing\
    |-- app.py                         # Streamlit user interface
    |-- parser.py                      # NER, regex, skills and matching logic
    |-- api.py                         # Optional FastAPI endpoints
    |-- elasticsearch_store.py          # Optional Elasticsearch storage/search
    |-- sample_job_description.txt      # Sample job description
    +-- sample_resumes.json             # Three sample candidate records
```

File	Purpose	Required for Streamlit run?
app.py	Displays input fields, upload control, ranking table and extracted information.	Yes
parser.py	Contains parse_resume, extract_skills and match_candidates.	Yes
sample_job_description.txt	Separate copy of sample job description.	No; app.py defines a default job description in the source.
sample_resumes.json	Separate copy of three sample resumes.	No; app.py defines the same sample candidates in the source.
api.py	Optional REST API for parse, rank and index operations.	No
elasticsearch_store.py	Optional search index and candidate lookup helper.	No

Why are there two copies of sample data?

The Streamlit app.py defines SAMPLE_JOB and SAMPLE_CANDIDATES directly in the code. The .txt and .json files are companion data files for inspection, editing or future extensions; the current app does not automatically load those companion files.

3.1 Software requirements

Package / model	Purpose	Classroom status
Python 3.12 virtual environment nlp-env	Isolated package environment shared by the NLP projects.	Already created and reused.
spaCy	NLP processing and NER pipeline.	Installed for Project 3.
en_core_web_sm 3.8.0	English pretrained spaCy pipeline.	Downloaded and validated.
Streamlit	Runs the browser-based interface.	Already installed during Project 2.
Pandas	Builds the candidate ranking table.	Used by the app; normally installed as a Streamlit dependency.
FastAPI / Uvicorn	Optional REST API service.	Only needed for the optional API extension.
Elasticsearch Python client	Optional external index and search feature.	Only needed for Elasticsearch extension.

3.2 Commands used to install and verify spaCy

```
python -m pip install --default-timeout=1000 --retries 10 --prefer-binary spacy
python -m spacy download en_core_web_sm
python -m spacy validate
python -c "import spacy; nlp = spacy.load('en_core_web_sm'); print('spaCy model loaded successfully!')"
```

The classroom terminal confirmed spaCy 3.8.18 and en_core_web_sm 3.8.0, and printed “spaCy model loaded successfully!”. Streamlit was already installed from Project 2, so it did not need to be reinstalled.

4. Step-by-step execution in VS Code

4.1 Open the correct folder

Open the nlp_projects workspace in VS Code. Select Terminal > New Terminal. The terminal should be PowerShell. Navigate to the resume_parsing directory from the workspace root:

```
cd C:\Users\spriy\nlp_projects\resume_parsing
dir | Select-Object Name, Length
```

The classroom directory listing showed app.py, parser.py, api.py, elasticsearch_store.py, sample_job_description.txt and sample_resumes.json.

4.2 Activate the shared virtual environment

The first activation attempt in PowerShell may be blocked by the Windows execution policy. The classroom solution changed the policy only for the current terminal process, then activated the environment.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
..\nlp-env\Scripts\Activate.ps1
```

Activation check

The terminal prompt should begin with (nlp-env), for example: (nlp-env) PS C:\Users\spriy\nlp_projects\resume_parsing>. If you are already in this folder and the prompt already shows (nlp-env), do not activate it again.

4.3 Check the packages and model

```
python -m pip show spacy streamlit
python -m spacy validate
python -c "import spacy; nlp = spacy.load('en_core_web_sm'); print('spaCy model loaded successfully!')"
```

4.4 Launch the Streamlit application

```
streamlit run app.py
```

Streamlit starts a local server and normally opens the browser at http://localhost:8501. If the browser does not open automatically, enter that address in the browser. Keep the terminal open while using the application.

4.5 Stop the application

```
Ctrl + C
```

Press Ctrl+C in the terminal running Streamlit to stop the local server. This does not delete or modify the project files.

4.6 The PowerShell path lesson

Path prefix	Meaning	Example
.\	Refer to an item inside the current directory.	cd .\resume_parsing (when the terminal is at nlp_projects).
..\	Move to the parent directory, then refer to a sibling folder.	From content_summarization, cd ..\resume_parsing.

In the classroom session, the terminal was already at C:\Users\spriy\nlp_projects. The command cd .. \resume_parsing looked for C:\Users\spriy\resume_parsing and failed. The correct command from the workspace root was cd .\resume_parsing.

5. Using the Streamlit application

5.1 Application input controls

Interface element	What it does
Job description	A text area initially filled with the sample Python-developer job description.
Upload candidate resumes	Allows one or more .txt files to be uploaded. The current app does not accept PDF or DOCX.
Built-in sample message	When no files are uploaded, the app uses three built-in sample resumes.
Parse resumes and rank candidates	Starts skill extraction, parsing, score calculation and candidate ranking.

5.2 First demonstration

For the first run, no file upload is required. Keep the built-in sample job description and click “Parse resumes and rank candidates”. The application detects the job-description skills, processes Candidate_A, Candidate_B and Candidate_C, creates the ranking table, and shows extracted information in expandable sections.

Result area	Displayed information
Required skills detected	Comma-separated list of skills identified in the job description.
Candidate ranking	Rank, candidate ID, detected name, match percentage, matched skills and missing skills.
Extracted resume information	The parsed names, emails, phones, skills, organizations/experiences and education terms for each candidate.

5.3 Sample job description

```
We are looking for a Python developer with experience in machine learning, natural language processing,
SQL, pandas, and scikit-learn. Experience with FastAPI and Docker is an advantage.
```

5.4 Sample candidates

Candidate ID	Name in sample text	Education	Key skill profile
Candidate_A	Ananya Rao	B.Tech Computer Science	Python, machine learning, NLP, SQL, pandas, scikit-learn, FastAPI
Candidate_B	Rahul Kumar	B.E. Information Science	Python, SQL, pandas, Docker, Java, data analysis
Candidate_C	Meera Nair	M.Sc. Data Science	Python, machine learning, scikit-learn, NLP, FastAPI, Docker

5.5 Expected score calculation for the supplied sample data

The default job description contains eight skills recognized by the configured vocabulary: Python, machine learning, natural language processing, SQL, pandas, scikit-learn, FastAPI and Docker. The following percentages are calculated from the sample resume text and the current SKILL_ALIASES vocabulary.

Candidate	Matched skills count	Calculation	Expected score
Candidate_A	7 of 8	7 / 8 × 100	87.5%
Candidate_C	6 of 8	6 / 8 × 100	75.0%
Candidate_B	4 of 8	4 / 8 × 100	50.0%

Interpretation

The expected ranking for the supplied default sample data is Candidate_A, Candidate_C, then Candidate_B. The score only measures overlap with the configured skill vocabulary. It should not be interpreted as a hiring recommendation or a measure of a person’s actual ability.

6. Complete source code

The following code listings are copied from the resume_parsing project files used for this manual. Keep the source files in the same project folder so that app.py can import parser.py.

6.1 app.py - Streamlit user interface

This file creates the browser interface, provides the sample data, accepts uploaded UTF-8 text files, calls the parser functions and displays a ranking table.

```
"""Streamlit demo for resume parsing and job-skill matching.

Run from this folder with:
    streamlit run app.py
"""

import json
from pathlib import Path
import pandas as pd
import streamlit as st
from parser import match_candidates, parse_resume, extract_skills

BASE_DIR = Path(__file__).resolve().parent

st.set_page_config(page_title="Resume Parser and Candidate Matcher", page_icon="📄", layout="wide")
st.title("Resume Parser & Candidate Matcher")
st.write("Extract candidate details with spaCy and regular expressions, then rank candidates by required-skill coverage.")

SAMPLE_JOB = """We are looking for a Python developer with experience in machine learning, natural language processing, SQL,
pandas, and scikit-learn. Experience with FastAPI and Docker is an advantage."""

SAMPLE_CANDIDATES = [
    {
        "candidate_id": "Candidate_A",
        "resume_text": """Ananya Rao\nEmail: ananya.rao@example.com\nPhone: +91 98765 43210\nEducation: B.Tech Computer
Science\nSkills: Python, machine learning, natural language processing, SQL, pandas, scikit-learn, FastAPI.\nExperience: Data
Science Intern at ABC Technologies.""",
    },
    {
        "candidate_id": "Candidate_B",
        "resume_text": """Rahul Kumar\nEmail: rahul.kumar@example.com\nPhone: 9876543210\nEducation: B.E. Information
Science\nSkills: Python, SQL, pandas, Docker, Java, data analysis.\nExperience: Software Engineer at Bright Systems.""",
    },
    {
        "candidate_id": "Candidate_C",
        "resume_text": """Meera Nair\nEmail: meera.nair@example.com\nPhone: +91-9988776655\nEducation: M.Sc. Data
Science\nSkills: Python, machine learning, scikit-learn, natural language processing, FastAPI, Docker.\nExperience: Research
Assistant at Data Research Lab.""",
    },
]

job_description = st.text_area("Job description", value=SAMPLE_JOB, height=130)

uploaded_files = st.file_uploader(
    "Upload candidate resumes as .txt files (optional). When none are uploaded, the three sample resumes are used.",
    type=["txt"], accept_multiple_files=True,
)

candidate_pairs = []
if uploaded_files:
    for file in uploaded_files:
        try:
            content = file.read().decode("utf-8-sig")
        except UnicodeDecodeError:
            st.error(f"Could not read {file.name}. Save the file as UTF-8 text and upload again.")
            continue
        candidate_pairs.append((file.name, content))
else:
    candidate_pairs = [(c["candidate_id"], c["resume_text"]) for c in SAMPLE_CANDIDATES]
    st.info("Using built-in sample resumes for the first demonstration.")

if st.button("Parse resumes and rank candidates", type="primary"):
    try:
        required_skills = extract_skills(job_description)
        if not required_skills:
            st.error("No configured skills were detected in the job description. Add terms such as Python, SQL, or machine
learning.")
        else:
            st.subheader("Required skills detected")
            st.write(", ".join(required_skills))
            ranked = match_candidates(job_description, candidate_pairs)
            table_rows = [{
                "Rank": index + 1,
                "Candidate": row["candidate_id"],
                "Detected name": row["candidate_name"],
                "Match score": f"{row['match_score'] * 100:.1f}%",
                "Matched skills": ", ".join(row["matched_skills"]),
                "Missing skills": ", ".join(row["missing_skills"]),
            } for index, row in enumerate(ranked)]
            st.subheader("Candidate ranking")
            st.dataframe(pd.DataFrame(table_rows), use_container_width=True, hide_index=True)

            st.subheader("Extracted resume information")
            for row in ranked:
                with st.expander(f"{row['candidate_id']} — {row['candidate_name']} ({row['match_score'] * 100:.1f}%)"):
                    st.json(row["parsed_information"])
    except Exception as exc:
        st.error(str(exc))
        st.info("Check that the spaCy model is installed in the active nlp-env environment.")

st.divider()
st.caption("Teaching note: this is a rule-assisted prototype. spaCy identifies general entities; the editable vocabulary
identifies skills. It is not a validated hiring or employment decision system.")
```

6.2 parser.py - NER, regex, skill extraction and matching

This is the core processing file. It loads spaCy, defines the skills vocabulary, extracts structured fields from each resume and calculates candidate-to-job skill coverage.

```
"""Resume information extraction and candidate-to-job skill matching.

The initial classroom implementation uses spaCy's en_core_web_sm model as
installed by the supplied environment guide. Set SPACY_MODEL=en_core_web_trf
when the transformer model is installed and you want to use it instead.
"""

import os
import re
from functools import lru_cache

SPACY_MODEL = os.getenv("SPACY_MODEL", "en_core_web_sm")

# Editable classroom vocabulary. A skill is extracted only when it appears in
# this list; add domain-specific skills before using the project with a class.
SKILL_ALIASES = {
    "python": ["python"],
    "java": ["java"],
    "c++": ["c++"],
    "c#": ["c#"],
    "sql": ["sql"],
    "machine learning": ["machine learning", "ml"],
    "deep learning": ["deep learning"],
    "natural language processing": ["natural language processing", "nlp"],
    "computer vision": ["computer vision"],
    "data analysis": ["data analysis", "data analytics"],
    "pandas": ["pandas"],
    "numpy": ["numpy"],
    "scikit-learn": ["scikit-learn", "sklearn"],
    "pytorch": ["pytorch", "torch"],
    "tensorflow": ["tensorflow"],
    "spacy": ["spacy", "spaCy"],
    "transformers": ["transformers", "hugging face"],
    "postgresql": ["postgresql", "postgres"],
    "mysql": ["mysql"],
    "mongodb": ["mongodb"],
    "fastapi": ["fastapi"],
    "flask": ["flask"],
    "django": ["django"],
    "html": ["html"],
    "css": ["css"],
    "javascript": ["javascript"],
    "react": ["react"],
    "git": ["git"],
    "docker": ["docker"],
    "aws": ["aws", "amazon web services"],
    "azure": ["azure", "microsoft azure"],
    "power bi": ["power bi"],
    "tableau": ["tableau"],
    "opencv": ["opencv", "open cv"],
}

EDUCATION_TERMS = [
    "b.e.", "b.e", "b.tech", "btech", "m.e.", "m.e", "m.tech", "mtech",
    "b.sc", "bsc", "m.sc", "msc", "mba", "ph.d", "phd", "bachelor",
    "master", "diploma",
]


@lru_cache(maxsize=1)
def load_nlp():
    import spacy
    try:
        return spacy.load(SPACY_MODEL)
    except OSError as exc:
        raise RuntimeError(
            f"spaCy model '{SPACY_MODEL}' is not installed. Run: "
            f"python -m spacy download {SPACY_MODEL}"
        ) from exc


def _contains_phrase(text_lower, phrase):
    """Match a phrase without matching it inside a longer alphanumeric word."""
    escaped = re.escape(phrase.lower())
    pattern = rf"(?<![a-z0-9]){escaped}(?![a-z0-9])"
    return re.search(pattern, text_lower) is not None


def extract_skills(text):
    text_lower = text.lower()
    found = []
    for canonical_skill, aliases in SKILL_ALIASES.items():
        if any(_contains_phrase(text_lower, alias) for alias in aliases):
            found.append(canonical_skill)
    return sorted(found)
```

```
def parse_resume(resume_text):
    if not resume_text or not resume_text.strip():
        raise ValueError("Resume text cannot be blank.")

    nlp = load_nlp()
    doc = nlp(resume_text)
    names = []
    organizations = []
    for entity in doc.ents:
        if entity.label_ == "PERSON" and entity.text not in names:
            names.append(entity.text)
        elif entity.label_ == "ORG" and entity.text not in organizations:
            organizations.append(entity.text)

    emails = sorted(set(re.findall(
        r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b",
        resume_text,
    )))
    # Deliberately permissive for classroom examples; validate country-specific
    # formats separately before using phone numbers in a real application.
    phone_candidates = re.findall(r"(?<!\w)(?:\+?\d[\d .()\-]{7,}\d)(?!\w)", resume_text)
    phones = sorted({re.sub(r"\s+", " ", value).strip() for value in phone_candidates})

    lower_text = resume_text.lower()
    education = [term for term in EDUCATION_TERMS if _contains_phrase(lower_text, term)]

    return {
        "names": names,
        "emails": emails,
        "phones": phones,
        "skills": extract_skills(resume_text),
        "experiences": organizations,
        "education": education,
    }


def match_candidates(job_description, candidate_resumes):
    """Rank (candidate_id, resume_text) pairs by required-skill coverage."""
    required_skills = extract_skills(job_description)
    if not required_skills:
        raise ValueError(
            "No skills from the configured skill vocabulary were found in the "
            "job description. Add the required terms to SKILL_ALIASES or edit the job description."
        )

    results = []
    for candidate_id, resume_text in candidate_resumes:
        parsed = parse_resume(resume_text)
        matched = sorted(set(parsed["skills"]) & set(required_skills))
        score = len(matched) / len(required_skills)
        results.append({
            "candidate_id": candidate_id,
            "candidate_name": parsed["names"][0] if parsed["names"] else "Not detected",
            "match_score": round(score, 4),
            "matched_skills": matched,
            "missing_skills": sorted(set(required_skills) - set(matched)),
            "parsed_information": parsed,
        })

    return sorted(results, key=lambda item: item["match_score"], reverse=True)
```

6.3 sample_job_description.txt

```
We are looking for a Python developer with experience in machine learning, natural language processing,
SQL, pandas, and scikit-learn. Experience with FastAPI and Docker is an advantage.
```

6.4 sample_resumes.json

The companion JSON file stores the same three sample resumes as structured records. The current app.py embeds sample candidate data directly and does not automatically read this JSON file.

```
[
  {
    "candidate_id": "Candidate_A",
    "resume_text": "Ananya Rao\nEmail: ananya.rao@example.com\nPhone: +91 98765 43210\nEducation: B.Tech Computer
Science\nSkills: Python, machine learning, natural language processing, SQL, pandas, scikit-learn, FastAPI.
\nExperience: Data Science Intern at ABC Technologies."
  },
  {
    "candidate_id": "Candidate_B",
    "resume_text": "Rahul Kumar\nEmail: rahul.kumar@example.com\nPhone: 9876543210\nEducation: B.E. Information
Science\nSkills: Python, SQL, pandas, Docker, Java, data analysis.\nExperience: Software Engineer at Bright Systems."
  },
  {
    "candidate_id": "Candidate_C",
    "resume_text": "Meera Nair\nEmail: meera.nair@example.com\nPhone: +91-9988776655\nEducation: M.Sc. Data
Science\nSkills: Python, machine learning, scikit-learn, natural language processing, FastAPI, Docker.\nExperience:
Research Assistant at Data Research Lab."
```

```
  }
]
```

7. Code walkthrough: how the parser works

7.1 Load spaCy only once

```
@lru_cache(maxsize=1)
def load_nlp():
    import spacy
    return spacy.load(SPACY_MODEL)
```

The lru_cache decorator stores the loaded NLP pipeline. This prevents the same model from being loaded again every time another resume is parsed during the same process. The actual parser also catches a missing-model error and reports the exact spaCy download command.

7.2 Match a phrase with word boundaries

```
escaped = re.escape(phrase.lower())
pattern = rf"(?<![a-z0-9]){escaped}(?![a-z0-9])"
return re.search(pattern, text_lower) is not None
```

This helper prevents a skill from being accidentally matched as part of a longer alphanumeric word. For example, a boundary check helps distinguish “java” from “javascript”. re.escape protects special characters in phrases such as c++ and c# before they are inserted into a regex pattern.

7.3 Extract configured skills

extract_skills converts the text to lowercase, loops through every canonical skill and its aliases, and records the canonical name if any alias is found. It returns a sorted list with each canonical skill listed once.

7.4 Parse one resume

Operation	Explanation
Blank input check	Raises ValueError if resume text is empty or contains only whitespace.
spaCy processing	nlp(resume_text) creates a Doc object and entity spans.
PERSON entities	Unique PERSON entity text is appended to names.
ORG entities	Unique ORG entity text is appended to organizations.
Email regex	Finds email-like patterns; results are de-duplicated and sorted.
Phone regex	Finds permissive phone candidates and normalizes repeated whitespace.
Education terms	Checks for configured degree terms using phrase-boundary matching.
Skills	Calls extract_skills on the resume text.
Return structure	Returns names, emails, phones, skills, experiences and education.

7.5 Match multiple candidates

```
required_skills = extract_skills(job_description)
matched = sorted(set(parsed["skills"]) & set(required_skills))
score = len(matched) / len(required_skills)
missing = sorted(set(required_skills) - set(matched))
```

For each resume, the code parses information, calculates the intersection between candidate skills and required skills, calculates the missing-skill set, and creates a result record. Finally, the results are sorted by match_score in descending order.

7.6 Display results in Streamlit

app.py converts the ranked result objects into table rows. It formats match_score as a percentage and joins the matched and missing skill names into readable strings. An expander for each candidate displays the full parsed_information object as JSON.

8. Optional extensions: FastAPI and Elasticsearch

These extensions are not required for the core lab

The successful classroom demonstration used Streamlit, spaCy and the local sample resumes. FastAPI and Elasticsearch are separate extension files. Use them only after the local parser is understood and the required packages/services are available.

8.1 api.py - FastAPI backend

The API file exposes endpoints for parsing one resume, ranking multiple candidates and optionally indexing a parsed resume. It is not called by the Streamlit app.

```
"""Optional FastAPI backend for resume parsing and candidate ranking.

Run from this folder with:
    uvicorn api:app --reload
Open http://127.0.0.1:8000/docs
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from parser import parse_resume, match_candidates

app = FastAPI(title="Resume Parser and Candidate Matcher API", version="1.0")


class ResumeRequest(BaseModel):
    resume_text: str = Field(min_length=5)


class CandidateInput(BaseModel):
    candidate_id: str
    resume_text: str = Field(min_length=5)


class RankingRequest(BaseModel):
    job_description: str = Field(min_length=5)
    candidates: list[CandidateInput]


class IndexRequest(BaseModel):
    candidate_id: str = Field(min_length=1)
    resume_text: str = Field(min_length=5)


@app.get("/")
def home():
    return {"message": "Resume Parser API is running."}


@app.post("/parse")
def parse(request: ResumeRequest):
    try:
        return parse_resume(request.resume_text)
    except Exception as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc


@app.post("/rank")
def rank(request: RankingRequest):
    pairs = [(item.candidate_id, item.resume_text) for item in request.candidates]
    try:
        return {"ranked_candidates": match_candidates(request.job_description, pairs)}
    except Exception as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc


@app.post("/index")
def index_resume(request: IndexRequest):
    """Optional Elasticsearch indexing endpoint; requires configured Elasticsearch."""
    try:
        from elasticsearch_store import index_candidate
        parsed = parse_resume(request.resume_text)
        index_candidate(request.candidate_id, request.resume_text, parsed)
        return {"indexed": True, "candidate_id": request.candidate_id, "parsed": parsed}
    except Exception as exc:
        raise HTTPException(status_code=503, detail=str(exc)) from exc
```

8.2 Run the optional FastAPI service

If FastAPI and Uvicorn are available in the active environment, open a terminal in resume_parsing and execute:

```
python -m pip show fastapi uvicorn
uvicorn api:app --reload
```

Open http://127.0.0.1:8000/docs to inspect and test the interactive API documentation. Stop the server with Ctrl+C. This API run was not part of the confirmed Streamlit execution; it is an optional next-stage exercise.

Endpoint	Method	Purpose
/	GET	Health/message response.
/parse	POST	Receives resume_text and returns parsed fields.
/rank	POST	Receives job_description and candidate records, then returns ranked candidates.
/index	POST	Attempts to index a parsed candidate in Elasticsearch; needs a configured reachable service and client package.

8.3 elasticsearch_store.py - optional storage and search

The module connects to an Elasticsearch service using ELASTICSEARCH_URL and, where required, ELASTICSEARCH_API_KEY. It creates an index named nlp_resumes, stores selected candidate fields and offers a multi-field search function. Elasticsearch must be installed and a reachable server must already be configured.

```
"""Optional Elasticsearch storage and search for parsed resumes.

Install: python -m pip install elasticsearch
Set ELASTICSEARCH_URL to your reachable Elasticsearch endpoint. For secured
clusters, also set ELASTICSEARCH_API_KEY (or adapt the client for your setup).
"""

import os
from elasticsearch import Elasticsearch

INDEX_NAME = "nlp_resumes"


def get_client():
    url = os.getenv("ELASTICSEARCH_URL")
    if not url:
        raise RuntimeError("Set ELASTICSEARCH_URL to your Elasticsearch endpoint first.")
    api_key = os.getenv("ELASTICSEARCH_API_KEY")
    if api_key:
        return Elasticsearch(url, api_key=api_key)
    return Elasticsearch(url)


def ensure_index(client):
    if not client.indices.exists(index=INDEX_NAME):
        client.indices.create(
            index=INDEX_NAME,
            mappings={
                "properties": {
                    "candidate_id": {"type": "keyword"},
                    "candidate_name": {"type": "text"},
                    "emails": {"type": "keyword"},
                    "phones": {"type": "keyword"},
                    "skills": {"type": "keyword"},
                    "experiences": {"type": "text"},
                    "education": {"type": "keyword"},
                    "resume_text": {"type": "text"},
                }
            },
        )


def index_candidate(candidate_id, resume_text, parsed):
    client = get_client()
    ensure_index(client)
    document = {
        "candidate_id": candidate_id,
        "candidate_name": parsed["names"][0] if parsed["names"] else "",
        "emails": parsed["emails"],
        "phones": parsed["phones"],
        "skills": parsed["skills"],
        "experiences": parsed["experiences"],
        "education": parsed["education"],
        "resume_text": resume_text,
    }
    client.index(index=INDEX_NAME, id=candidate_id, document=document, refresh=True)
    return document


def search_candidates(query, size=10):
    client = get_client()
    response = client.search(
        index=INDEX_NAME,
        size=size,
        query={
            "multi_match": {
                "query": query,
                "fields": ["candidate_name^2", "skills^3", "experiences", "education", "resume_text"],
            }
        },
    )
    return [hit["_source"] for hit in response["hits"]["hits"]]
```

8.4 Configure Elasticsearch (advanced)

```
python -m pip install elasticsearch
$env:ELASTICSEARCH_URL = "https://your-elasticsearch-host:9200"
$env:ELASTICSEARCH_API_KEY = "your-api-key"
```

These are placeholders, not working credentials. Replace them with the endpoint and credentials supplied by the Elasticsearch administrator. Never place real API keys in source code or share them in screenshots. The Elasticsearch extension was not required or verified during the local classroom demonstration.

9. Limitations and responsible interpretation

Current implementation behaviour	Why it matters / possible improvement
Skills come from SKILL_ALIASES only.	A skill absent from the dictionary is not detected. Extend the vocabulary and aliases for the course domain.
The score treats all detected job-description skills equally.	It does not distinguish essential from optional skills. A future version could separate required and preferred skills and weight them explicitly.
Words such as “advantage” are not interpreted as optional by the parser.	In the supplied job text, FastAPI and Docker appear as an advantage, but both are included in required_skills if they appear in SKILL_ALIASES.
The phone regex is intentionally permissive.	It may capture a number-like sequence that is not a valid phone number. Add country-specific validation before real use.
spaCy NER may miss or mislabel names and organizations.	Review the extracted result; consider structured patterns or human verification.
Only .txt upload is enabled.	PDF and DOCX extraction would require additional document-processing libraries; scanned documents may need OCR.
The first detected PERSON entity becomes candidate_name.	A person’s name elsewhere in the resume could be selected. Improve name-section logic or require candidate confirmation.
Education is keyword-based.	The output indicates detected degree terms, not verified credentials, dates or equivalence.
The score measures vocabulary overlap only.	It does not assess experience depth, project quality, communication, certifications or actual proficiency.

Ethical and classroom use

Use synthetic or consented sample resumes for classroom demonstrations. Do not use this prototype as the sole basis for hiring, rejection or employment decisions. Keep personal contact data private, review extraction errors and provide human oversight.

10. Troubleshooting guide

Problem / message	Likely cause	Action
ModuleNotFoundError: No module named spacy	spaCy is missing from the active virtual environment.	Activate nlp-env; run python -m pip install --default-timeout=1000 --retries 10 --prefer-binary spacy.
spaCy model en_core_web_sm is not installed	The Python package is installed but the separate language model is missing.	Run python -m spacy download en_core_web_sm.
Could not load model / OSError	The model install is incomplete or a different environment is active.	Check (nlp-env), run python -m spacy validate, then test spacy.load.
ModuleNotFoundError: streamlit	Streamlit is missing from the active environment.	Activate nlp-env and install streamlit.
ModuleNotFoundError: pandas	Pandas is missing from the active environment.	Run python -m pip install pandas.
No configured skills detected	The job description contains no terms from SKILL_ALIASES.	Use known terms such as Python, SQL, pandas or add new aliases to parser.py.
Could not read uploaded file	Uploaded text is not decodable as UTF-8.	Save the file as UTF-8 .txt and upload again.
Resume text cannot be blank	A blank resume was passed to parse_resume.	Add non-empty resume text.
Browser does not open	The Streamlit server started but browser launch did not occur.	Open http://localhost:8501 manually while the terminal is running.
Port 8501 is already in use	Another Streamlit process is already running.	Stop the old process with Ctrl+C or use the new port shown by Streamlit.
Elasticsearch endpoint error	The optional service URL or credentials are not configured.	Set ELASTICSEARCH_URL and, where required, ELASTICSEARCH_API_KEY; confirm that the service is reachable.

10.1 Classroom execution log

Step completed	Observed status from the classroom session
Navigated to resume_parsing.	The directory listing showed all six files: app.py, parser.py, api.py, elasticsearch_store.py, sample_job_description.txt and sample_resumes.json.
Activated the shared environment.	Terminal prompt showed (nlp-env).
Checked packages.	Streamlit was present; spaCy was initially missing.
Installed spaCy and its model.	spaCy 3.8.18 and en_core_web_sm 3.8.0 were validated.
Tested the model load.	The terminal printed “spaCy model loaded successfully!”.
Started Streamlit.	The Resume Parser & Candidate Matcher interface opened at localhost:8501.
Used built-in sample resumes.	The user confirmed that candidate details and matching percentages were displayed successfully.
Stopped the server.	Ctrl+C was used to end the Streamlit process.

11. Experiments and observation worksheet

11.1 Recommended experiments

Experiment	Change to make	What to observe
A. Baseline sample	Use the default job description and three sample resumes.	Record rank, detected name, score, matched skills and missing skills.
B. Add a skill	Add a skill already in SKILL_ALIASES to the job description.	Observe how the denominator and candidate scores change.
C. Remove a requirement	Remove SQL or Docker from the job description.	Observe which candidates gain or lose relative coverage.
D. Add an alias	Add an alternative phrase to a skill in SKILL_ALIASES.	Test whether the new phrase maps to the canonical skill.
E. Test boundaries	Compare Java with JavaScript and c++ with unrelated text.	Check that phrase boundaries avoid partial-word false matches.
F. Upload custom resumes	Create two or three UTF-8 .txt files and upload them.	Compare file-upload behaviour with the built-in samples.
G. Missing skill vocabulary	Use a job description containing an unsupported skill only.	Observe the no-skills-detected error and explain why it occurs.
H. Incomplete resume	Remove the email or phone field from a sample resume.	Observe empty extraction arrays and discuss missing data.

11.2 Student observation worksheet

Observation item	Candidate A	Candidate B	Candidate C
Detected name	_____	_____	_____
Email detected?	Yes / No	Yes / No	Yes / No
Phone detected?	Yes / No	Yes / No	Yes / No
Education terms	_____	_____	_____
Skills detected	_____	_____	_____
Matched skills	_____	_____	_____
Missing skills	_____	_____	_____
Match score	_____ %	_____ %	_____ %

11.3 Questions for result discussion

- Did spaCy identify the candidate’s name correctly? Was the first PERSON entity the candidate?
- Which email and phone patterns were found? Were any patterns incorrectly captured?
- Did the skills list contain every skill in the resume? Which missing skills were absent from SKILL_ALIASES?
- Does the score distinguish a required skill from a preferred skill?
- Would two candidates with identical skill coverage necessarily be equally suitable? Why not?

12. Student exercises

1. Explain the difference between Named Entity Recognition and regular-expression extraction using one field from a resume.
2. Add two technical skills and their aliases to SKILL_ALIASES. Test the new terms in both a resume and a job description.
3. Create a new UTF-8 .txt resume with a name, email, phone number, education term, organization and skills. Upload it through Streamlit.
4. Change the job description to require only Python, SQL and pandas. Recalculate the candidate scores manually and compare with the application.
5. Explain why the current parser treats FastAPI and Docker as requirements when they appear in the default job description, even though they are described as advantages.
6. Modify the parser to return all entity labels for inspection, then compare PERSON and ORG with another label found by spaCy.
7. Explain why a phone number detected by regex should be validated before being used in an application.
8. Describe the additional steps needed to support PDF and DOCX resumes. Distinguish text extraction from OCR for scanned resumes.

13. Viva questions with answer guidance

1. What is resume parsing?

Answer guidance: Converting unstructured resume text into structured fields such as names, contact details, education, organizations and skills.

2. What is NER?

Answer guidance: Named Entity Recognition identifies text spans and assigns labels such as PERSON or ORG.

3. Which NLP library is used?

Answer guidance: spaCy.

4. Which English model is used in the executed project?

Answer guidance: en_core_web_sm, version 3.8.0 in the classroom setup.

5. What is the role of spacy.load()?

Answer guidance: It loads the pretrained NLP pipeline so text can be processed.

6. What does doc.ents contain?

Answer guidance: Entity spans detected by the spaCy pipeline.

7. Which entity labels are collected by the parser?

Answer guidance: PERSON and ORG.

8. Why are regular expressions used?

Answer guidance: They are useful for locating structured patterns such as email addresses and phone-number candidates.

9. What does re.findall() do?

Answer guidance: Returns all non-overlapping matches of a regex pattern in a string.

10. What is SKILL_ALIASES?

Answer guidance: An editable dictionary of canonical skill names and alternative forms used for skill detection.

11. Why convert text to lowercase?

Answer guidance: To make matching independent of capitalization.

12. What does re.escape() do?

Answer guidance: Escapes characters that have special meaning in regex patterns.

13. What is a Python set?

Answer guidance: A collection of unique elements; sets support intersection and difference operations.

14. What does the & operator do for sets?

Answer guidance: Returns the intersection: items present in both sets.

15. How is the match score calculated?

Answer guidance: Number of matched required skills divided by the number of detected required skills.

16. Why is the result multiplied by 100 in the UI?

Answer guidance: To display the fractional score as a percentage.

17. What does sorted(..., reverse=True) do here?

Answer guidance: Orders candidate results from highest score to lowest score.

18. What is lru_cache used for in load_nlp()?

Answer guidance: It reuses the loaded spaCy model instead of loading it repeatedly.

19. Does this app parse PDF and DOCX resumes?

Answer guidance: No. The current file uploader accepts .txt files only.

20. Is the match score a hiring decision?

Answer guidance: No. It measures configured skill overlap only and is not a validated employment decision system.

14. Faculty teaching plan and assessment

Teaching block	Suggested duration	Faculty activity / learner outcome
Problem and workflow	10 minutes	Introduce the resume parsing problem and show the input-to-ranking workflow.
NER and regex concepts	20 minutes	Explain PERSON / ORG entities, email regex and phone-pattern caveats.
Skills and set matching	20 minutes	Demonstrate the skill aliases, set intersection, missing-skill set and score formula.
Environment and execution	15 minutes	Verify nlp-env, spaCy, en_core_web_sm, and launch Streamlit.
Hands-on application	25 minutes	Students process sample resumes and upload custom .txt resumes.
Code walkthrough	20 minutes	Trace app.py to parser.py and explain the parsing and ranking functions.
Discussion and assessment	10 minutes	Discuss limitations, ethical use and viva questions.

14.1 Suggested assessment rubric

Criterion	Marks	Evidence
Environment and application execution	10	Activates the shared environment and launches the Streamlit app.
NER and regex understanding	20	Explains PERSON / ORG extraction, email and phone matching.
Skill extraction	20	Explains canonical skills, aliases and how to extend the vocabulary.
Matching logic	20	Manually computes matched / missing skills and match score.
Testing and interpretation	20	Tests custom resumes and explains ranking and limitations.
Code explanation and viva	10	Explains the main functions and answers conceptual questions.

15. References and project alignment

Primary project specification: NLP Projects.pdf, “Project 3: Resume Parser & Candidate Matcher (NER & Information Extraction)” and its “Step By Step Implementation” section. The specification identifies NER, regex contact extraction, skill matching, candidate ranking and optional Elasticsearch/FastAPI integration.

Environment basis: NLP Environment Setup & Duplicate Question Detector Guide (Revised). The shared nlp-env setup is retained for the resume parser project; the completed classroom run installed spaCy and downloaded en_core_web_sm.